

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 08 · LECTURE NOTES

Randomised Rounding

Turning a fractional solution into a real one.

MODULE OUTCOMES

Explain randomised rounding and why it analyses cleanly.

Derive the set cover guarantee.

Explain how feasibility is restored.

Explain dependent rounding and when it is needed.

Apply rounding to a problem of your own.

1 The technique

A FRACTIONAL VALUE IS A PROBABILITY, AND THAT MAKES THE ANALYSIS ONE LINE

Solve the linear programming relaxation of the integer program, then include each item independently with probability equal to its fractional value. That is the whole method, and it is almost too simple to be a technique.

The expected cost is exactly the LP value, by linearity of expectation (Module 07 §1) — **which is why this analyses so cleanly: no independence assumption is needed for the cost bound at all**, only for the feasibility analysis.

And the LP value is a lower bound on the integer optimum, because the relaxation admits every integral solution (CSCE 669 Module 04 §1's weak duality) — **so the expected cost is at most the optimum**, which is the entire mechanism of approximation analysis (CSCE 669 Module 10 §1).

Feasibility, however, holds only in expectation — the rounded solution may violate constraints — and **restoring it is where the approximation factor comes from (§2). The cost analysis is free and the feasibility analysis is the work**, which is the right expectation to have before starting.

SET COVER: cover every element with the fewest sets.
 The LP relaxation gives a fractional x_S for each set.

ROUND: include set S with probability x_S , independently.

COST: $E[\text{cost}] = \sum_S c_S \cdot x_S = \text{exactly the LP value.}$
 Done, by linearity.

COVERAGE: for an element e whose fractional coverage is at least 1,

$$P(e \text{ uncovered}) = \prod_{S \text{ containing } e} (1 - x_S)$$

$$\leq \exp(- \sum_{S \text{ containing } e} x_S)$$

$$\leq e^{-1}$$
 using $1 - z \leq e^{-z}$ and the fact that the fractional coverage sums to at least 1.

So each element is covered with probability $1 - 1/e$.
 NOT GOOD ENOUGH -- a constant fraction of elements will be left uncovered.

THE FIX: repeat the whole rounding $\ln(4n)$ times independently and take the UNION of all chosen sets.
 $P(e \text{ uncovered after } k \text{ rounds}) \leq e^{-k}$
 Set $k = \ln(4n)$: $P(\text{ANY element uncovered}) \leq n/(4n) = 1/4$, by the union bound (Module 02 section 3).
 And $E[\text{cost}] = k \cdot \text{LP} = O(\log n) \cdot \text{OPT}$.

SO: an $O(\log n)$ -approximation -- which CSCE 669 Module 10 and CSCE 637 Module 11 both say is OPTIMAL. The trivial algorithm is the best one.

2 Restoring feasibility

Method	How it works	What it costs
Repeat and union	Round several times independently and take the union of the selections.	A logarithmic factor in the cost. Set cover (§1).
Scale up first	Round with probability $\min(1, \alpha x)$ for some $\alpha > 1$.	A factor of α in the cost, and it improves the feasibility probability correspondingly. The simplest, and usually tried first.
Repair afterwards	Round, then greedily fix whatever constraints are violated.	Bound the repair cost separately , which is sometimes easy and sometimes the whole difficulty.
Conditioning	Round variables one at a time, conditioning each choice on the remaining feasibility.	Harder to analyse and frequently gives a tighter ratio — and it is also the route to derandomisation (Module 12 §1).
Dependent rounding	Round in a way that preserves the constraints exactly throughout.	§3. The best option when it applies.

WHY THE LP BOUND IS WHAT MAKES ANY OF THIS WORK

The approximation ratio is proved against the LP value rather than against the optimum — which you cannot compute, since computing it is the NP-hard problem you are avoiding.

And the LP value bounds the optimum by weak duality (CSCE 669 Module 04 §1), **so a ratio established against the LP is automatically a ratio against the optimum.**

Which means the *integrality gap* of the relaxation limits what any rounding scheme can possibly achieve (CSCE 669 Module 10 §2) — **set cover's LP has a logarithmic integrality gap, so no rounding of that LP can beat $O(\log n)$, however clever.**

So the first question about a rounding scheme is the integrality gap of the relaxation — **it tells you the best ratio available before you design anything, and beating it requires a stronger relaxation rather than a better rounding** (a semidefinite one, or additional valid inequalities, which is CSCE 669 Module 09's formulation strength).

3 Dependent rounding

DEPENDENT ROUNDING PRESERVES CONSTRAINTS EXACTLY WHILE KEEPING THE MARGINALS

Independent rounding of a constraint like 'choose exactly k items' will essentially never choose exactly k — the count is a binomial variable, concentrated near k and almost never equal to it. So independence is sometimes the problem rather than the tool.

Dependent rounding: pick two fractional variables, move one up and the other down by the same amount until one of them reaches 0 or 1, preserving their sum. Repeat until every variable is integral.

Each variable's marginal probability of ending at 1 is exactly its original fractional value — so the expected-cost analysis of §1 still works unchanged — and the sum constraint is satisfied exactly rather than in expectation.

And the variables become *negatively correlated* rather than independent, which means Chernoff bounds still apply (Module 02 §2's negative association fallback) — so you keep the concentration as well as the feasibility. That is the property that makes the technique genuinely useful rather than merely feasible, and it is why ad allocation systems use it (§4).

4 Using it

- 1 · Write the integer program, then relax the integrality constraints (CSCE 669 Module 09 §1).
- 2 · Compute the integrality gap, or look it up. It bounds what you can achieve — and this is the step to do first and the one most often skipped, because it can tell you that your intended improvement is impossible.
- 3 · Round independently and compute the expected cost by linearity. One line, and it will equal the LP value.
- 4 · Check feasibility and choose a repair from §2's menu. This is the work, and the choice determines the approximation factor.
- 5 · And if a constraint must hold exactly rather than approximately, use dependent rounding (§3) — which is usually the case for budget, capacity, and cardinality constraints in a real system.

WHERE ROUNDING IS USED IN PRACTICE

Facility location, scheduling, and network design — the classical approximation applications, and **the ratios obtained by LP rounding are frequently the best known** for these problems.

Ad allocation and matching. Dependent rounding of a linear program is how large-scale ad serving allocates impressions to advertisers, because the budget constraints must hold exactly — you cannot overspend a budget in expectation.

Routing and load balancing, where the fractional solution is a flow (CSCE 669 Module 11 §1) and the rounding produces integral paths — which is how multi-commodity routing is approximated.

And inside integer programming solvers themselves — rounding heuristics are how a solver finds an early incumbent solution (CSCE 669 Module 09 §1), **which is what makes branch-and-bound's pruning effective at all**. So the technique is both an approximation method and a component of exact solvers, which is unusual.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Williamson & Shmoys — chapters 1 and 5 (free PDF)	Randomised rounding, free in full — the set cover derivation of §1 and the feasibility methods of §2.
Raghavan & Thompson — Randomized Rounding (free)	The originating paper , in the routing setting of §4.
Gandhi, Khuller, Parthasarathy & Srinivasan — Dependent Rounding (free)	§3's technique , with the negative correlation proved.
Motwani & Raghavan — chapter 4	The same material from the randomised-algorithms side , with the Chernoff analysis explicit.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Solve a set cover LP** and round it independently. Report the cost and how many elements were uncovered.
- 2 **Implement the repeat-and-union fix** and verify the $O(\log n)$ cost and the coverage.
- 3 **Measure the achieved ratio** against the LP bound on random instances.
- 4 **Compute the integrality gap** of the set cover LP on a small instance.

- 5 **Implement scale-up rounding** and plot the ratio against α .
- 6 **Round a 'choose exactly k' constraint independently** and report how often you get exactly k.
- 7 **Implement dependent rounding** and confirm the constraint holds exactly and the marginals are preserved.
- 8 **Verify the negative correlation** empirically.
- 9 **Apply the five-step procedure** to a problem of your own.
- 10 **Compare your rounded solution against a solver's exact answer** on instances small enough to solve.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Explain randomised rounding and why the cost analysis needs no independence.
- 2 Why is the ratio proved against the LP rather than the optimum?
- 3 Derive the set cover guarantee in full.
- 4 Why is the repeat-and-union step needed, and what does it cost?
- 5 Name five ways to restore feasibility.
- 6 What limits any rounding of a given LP, and what is needed to beat it?
- 7 Why is independent rounding wrong for an exact cardinality constraint?
- 8 Describe dependent rounding and name its three preserved properties.
- 9 Give the five-step procedure and say which step comes first.
- 10 Name four practical applications.